Lab drill: diagnose it, then say it

> Synthetic data. Fictional lending simulation. Not affiliated with any lender.

A sheet to work through at the keyboard, in order. Each exercise is **Do → See → Diagnose → Say**. "Say" is the sentence you would use in an interview. Total time first pass: about 50 minutes. Second pass: skip to the **Cold drill** at the end and do it without the answers.

Setup: `./start.sh`, open http://127.0.0.1:8510, sidebar → **Reset the lab**. Keep http://127.0.0.1:8010/docs open in a second tab. The three partners are **alder**, **birch** and **cedar**, keys `demo-key-alder-0001`, `demo-key-birch-0002`, `demo-key-cedar-0003`.

The diagnosis habit, every time, in this order:

1. **Which status class?** 4xx = the caller's request, 5xx = our code, 2xx-but-slow = capacity.
2. **What scope?** One partner or all? One endpoint or all? Since when?
3. **Find the request ID** in the log and look at the exact request.
4. **Reproduce** it (playground or curl), then fix, then say what you tell the partner.

---

1. The happy path, and the request ID

Do: API playground → example *Create customer* → Send. Then *Create application* → Send.

See: `201` with a customer `id`, then `201` with `"status": "approved"` and an `X-Request-ID` header. "What the API recorded" shows the log row.

Diagnose: nothing is wrong. Note the three things every response carries: a status code, a stable error `code` when it fails, and a request ID.

Say: "Every response carries a request ID that ties the partner's call to one row in our log, so a support ticket starts from a fact, not a description."

2. Fault B: the partner sends bad JSON (422)

Do: Fault injection → **B. Invalid application payloads** on, partner birch → *Generate traffic* (or playground: remove `product` from the body and Send).

See: some requests return `422 validation_error` with a `details` list naming every wrong field at once. Others from the same partner still succeed.

Diagnose: 4xx, one partner, only `POST /applications`, intermittent by request not by time. Pydantic rejected the body before our code ran. The partner's integration builds some payloads wrongly (missing field, string where an int should be, unknown product).

Say: "A 422 is the contract doing its job: the JSON was understood but doesn't fit the schema. I'd send the partner the `details` array and the schema example from `/docs`, and I'd ask what changed on their side on that date."

3. Fault C: IDs that don't exist, or aren't theirs (404)

Do: Fault injection → **C. Missing customer or application** on, birch → generate traffic. Then in the playground, as alder, `GET /customers/{id}` for a customer that belongs to cedar.

See: `404 customer_not_found` / `application_not_found`. The cross-partner read is also 404, not 403.

Diagnose: 4xx, client side. Either the partner stores IDs wrongly, or it is retrying with an ID from a different environment (sandbox vs live is the classic).

Say: "A partner can't see another partner's customers, and we return 404 rather than 403 so the API doesn't confirm that the ID exists. For the partner it's usually a sandbox/live mix-up."

4. Fault A: the key is revoked (401)

Do: Fault injection → **A. Authentication failure** on, cedar → *Send POST /customers as cedar*. Switch it off, send again.

See: `401 api_key_revoked`, then `201`.

Diagnose: 4xx, one partner, every endpoint, from one moment. Not a bug: a key state. Check rotation, check whether their old key was disabled on schedule, check they deployed the new key everywhere (a stale key on one server is "intermittent 401s").

Say: "All-endpoints 401 from one partner since a point in time is a key problem, not a code problem. First question: did we rotate, and did they deploy the new key to every instance?"

5. Fault F: rate limiting (429)

Do: Fault injection → **F. Simulated rate limiting** on, alder (20 per 60 s) → generate traffic quickly, or send the playground request 25 times.

See: after 20, `429 rate_limited` with a `Retry-After` header. The rest of the minute fails.

Diagnose: 4xx, one partner, time-clustered. Look at their request rate against the limit. The usual cause: a client that retries immediately after a timeout, with no backoff, so one slow response turns into a storm.

Say: "429 with Retry-After is us protecting the other partners. The fix is on both sides: they add exponential backoff with jitter and an idempotency key so retries can't double-submit; we check whether their limit is right for their volume."

6. Fault D: our scoring service throws (500)

Do: Fault injection → **D. Server-side exception** on, all partners, every 4th → generate traffic. Incident investigation → section 4 → click a failed row.

See: every fourth `POST /applications` returns `500 server_exception`; other endpoints are fine; all partners are affected.

Diagnose: 5xx = ours. All partners, one endpoint, a fixed pattern: a downstream dependency, not a partner. Reproduce with the request ID, read the trace, add a timeout and a circuit breaker so a dead dependency returns a clear 503 instead of hanging or 500ing.

Say: "A 5xx is never the partner's fault and I say so in the first message. Acknowledge, scope, next update time, then the fix and a short post-mortem."

7. Fault E: nothing fails, everything is slow

Do: Fault injection → **E. Increased latency** on, birch, 250 ms → generate traffic → Overview or Broker monitoring.

See: status codes unchanged; p95 latency for birch jumps.

Diagnose: status-code dashboards show nothing. Only latency percentiles per partner catch it.

Say: "This is why I watch p95 by partner and alert on change, not on level. An integration can be 100% 2xx and still be broken for the partner's customers."

8. Fault H: the webhook never arrives

Do: Fault injection → **H. Webhook delivery failure** on, cedar → create an application as cedar → section *Webhook deliveries* → then *Redeliver failed webhooks*.

See: the application is `201`, but its webhook delivery is recorded as failed (receiver 503). Redelivery succeeds once the fault is off.

Diagnose: the partner says "we never got the approval". The `webhook_deliveries` table is the audit trail: we tried, their endpoint said 503, we retried. Signed payloads, retries with backoff, idempotent

event IDs, and a replay button are the whole design.

Say: "Webhooks need a delivery log the partner can see. Then 'we never got it' becomes 'your endpoint returned 503 at 14:02, here are the three retries, click replay'."

9. The release story: v1.0.0 → v1.1.0 → v1.1.1

Do: Release simulation, exactly as in DEMO_SCRIPT.md: baseline traffic → activate v1.1.0 → same traffic → step 5 table → step 6 grid → read the rule's source → apply patch v1.1.1 → same traffic again.

See: refusals 4.0% → 27.3% (+23.3 pts, 95% interval +15.5 to +31.1). The 25k–50k band is 100% refused for all three products; the release notes said one product. `rules.py`: the `hotfix` branch checks the product, the other branch doesn't. After the patch: eligible products back to baseline, invoice finance still enforced.

Diagnose: 4xx (422) so it looks like the partners' fault, but it started at a release and hit every partner: that pattern means us. Same seed, same requests, so runs match request by request. The fix is the one-line bug, not a rollback that would also remove the intended control.

Say: "The notes were the claim; the log was the evidence. I compared identical traffic before and after, found the band and products affected, read the rule, and shipped the smallest patch that kept the intended control. Then I told partners what changed and for whom."

10. The data-science minute

Do: Data science → sections 3 and 4. Broker monitoring → sections 2, 4, 5.

See: naive −18.0 pts vs adjusted −11.0 vs true −12.2; the A/B test +5.5 pts (interval +3.9 to +7.2). Broker monitoring: a control chart flags Meridian on day 10; before-vs-during −10.9, difference-in-differences −11.5, true −9.6.

Say: "The naive number is a correlation inflated by application size. The randomised test is causal. For a partner-level change I'd use the other partners as the control, which is what the difference-in-differences does here."

Cold drill (no answers in front of you; answers below)

1. Alder reports "random 401s since this morning". Three questions you ask, in order.
2. Every partner's `POST /applications` is failing one time in four with 500. Whose fault, first message to partners, first thing you check.
3. Birch: 100% 2xx, but their product team says applicants are abandoning. What do you look at?
4. Refusals jumped from 4% to 27% an hour after a release. Walk through the diagnosis to the fix.
5. Cedar says they never received an approval webhook. What do you open and what do you say?
6. You need to change `amount` from pounds to pence. How do you ship it?

Answers

1. Did we rotate their key (and when)? Did they deploy the new key to every instance? Is it every endpoint or one? "Random" 401s are usually one stale server.
2. Ours (5xx). "We've identified an issue affecting application submissions since HH:MM, it is on our side, your data is safe, next update in 30 minutes." Then the scoring-service dependency and the request IDs of the failures.
3. Latency percentiles per partner and timeouts to downstreams; then the funnel from application start to submit. A slow API drops applicants without a single error.
4. Scope (all partners, one endpoint, since the release) → compare identical traffic before/after → which band and products → read the rule → patch the branch that skips the product check → rerun the same traffic → eligible back to baseline, intended rule still on → release note.

5. The webhook delivery log for that application: attempts, their endpoint's status, retries. "Your endpoint returned 503 at 14:02; we retried three times; I've replayed it now."
6. Additive first: `amount_pence` alongside `amount` in v1, documented, with a deprecation date; migrate partners one by one; remove `amount` in v2. Never change the meaning of a live field.